

Fintech AI Assistant - Final Project

1. Project Overview

In this project, you will build a comprehensive **Fintech AI Assistant** that integrates multiple AI frameworks to provide financial insights, portfolio analysis, and personalized recommendations.

This project demonstrates how different LangChain-based components can work together to build a domain-specific AI assistant.

Core Components:

- **LangChain:** To manage conversational flow and tool calling
- **RAG (Retrieval-Augmented Generation):** For context-based financial knowledge retrieval
- **OpenAI Models:** To generate responses using GPT-based reasoning
- **Streamlit:** For building an interactive user interface
- **SQLite:** To manage transaction and portfolio data

This project brings together all the core concepts learned during the training program, combining AI reasoning, retrieval, and interaction layers into one unified assistant.

2. Project Structure

We follow a modular architecture to maintain clear separation of concerns. Below is the recommended directory structure:

```
fintech_app/
├── app.py # Main Streamlit application
├── financial_knowledge.txt # RAG knowledge base
├── requirements.txt # Dependencies
|
├── data/
|   ├── finance_data.db # SQLite database
|   └── chroma_db/ # Vector store for embeddings
|
├── utils/
|   ├── init.py
|   ├── rag.py # Retrieval-Augmented Generation logic
|   ├── tools.py # Tool definitions (SQL, API, RAG)
|   └── database.py # Database setup and management
|
└── project_description.ipynb # Detailed documentation notebook
```

3. Database Setup

A pre-configured SQLite database (finance_data.db) is included in the project with the following tables:

- **users:** Contains user details and associated email IDs
- **portfolio:** Stores portfolio investment and performance data

You can extend the database with tables such as **transactions** for detailed expense tracking.

To reinitialize the database:

```
python utils/database.py
```

4. Core Concepts and System Flow

The Fintech AI Assistant handles different types of financial queries using an integrated architecture.

The query flow across modules is as follows:

4.1 Overview of the Query Flow

1. **Query Submission:** User submits a question through Streamlit.
 2. **Processing:** LangChain and RAG handle query classification and retrieval.
 3. **Tool Selection:** Appropriate tools (RAG, SQL, or API) are invoked.
 4. **Response Formulation:** Retrieved data is passed to the LLM for response generation.
 5. **Presentation:** Streamlit displays the structured output.
-

5. Query Categories and Processing

5.1 Knowledge Retrieval (Using RAG)

Example Queries:

- “Explain the Rule of 72.”
- “What is diversification in investing?”
- “What are the benefits of index funds?”

Flow:

1. User submits query.
2. System retrieves relevant content from financial_knowledge.txt.
3. GPT model summarizes and explains the retrieved text.

Key Implementation Points:

- Use Chroma for vector embeddings
 - Use LangChain retriever and history-aware chain for context
-

5.2 SQL-Based Analysis

Example Queries:

- “Show my investment portfolio performance.”

- “Calculate my total spending by category.”
- “Retrieve my monthly returns.”

Flow:

1. Query routed to SQL agent.
 2. The tool executes queries on `finance_data.db`.
 3. The result is formatted and returned to the user.
-

5.3 Real-Time Financial Data

Example Queries:

- “What’s the current price of AAPL?”
- “Show today’s market summary.”

Flow:

1. Query routed to the `yfinance` tool.
 2. Data fetched and formatted for display.
-

6. Streamlit Interface Implementation

The Streamlit app provides two main user roles:

1. **User Mode:** Allows chatting, viewing portfolio, and analyzing spending.
2. **Admin Mode:** Allows uploading new financial knowledge documents for RAG.

The interface includes:

- Chat-based interaction with message history
- Sidebar for conversation management
- Admin upload panel for updating knowledge base

To run the Streamlit app:

```
streamlit run app.py
```

7. Implementation Summary

7.1 RAG Integration (rag.py)

Handles document loading, chunking, vectorization, and retrieval.

Key Features:

- Supports .txt and .pdf files
 - Uses RecursiveCharacterTextSplitter for chunking
 - Embeddings stored in Chroma vector database
-

7.2 Tool Integration (tools.py)

Defines tools for:

- Retrieving portfolio data from SQLite
 - Fetching live stock data
 - Accessing knowledge base through RAG
-

7.3 Database Handling (database.py)

Handles database initialization, schema creation, and sample data insertion.

8. Sample Queries for Testing

8.1 Knowledge Queries (RAG)

- “What is diversification in investing?”
- “Explain the Rule of 72.”
- “What is Zero-Based Budgeting?”

8.2 SQL Queries

- “Show my portfolio details.”
- “Summarize my investments.”

8.3 Stock Data Queries

- “Get AAPL stock price.”
 - “Show today’s NIFTY50 performance.”
-

9. Conclusion

This project brings together **LangChain**, **Streamlit**, **OpenAI**, and **RAG** into a cohesive Fintech Assistant application.

By completing this project, you demonstrate the ability to:

1. Use **LangChain** for multi-tool query orchestration.
2. Implement **RAG** for document-based knowledge retrieval.
3. Integrate **databases and APIs** for financial insights.
4. Build an **interactive AI assistant UI** using Streamlit.

This project highlights the end-to-end integration of **LLMs with financial data systems**, showcasing practical use cases in **Fintech automation and AI assistance**.